Лабораторная работа №3
Проектирование архитектуры программного продукта

****Дисциплина:**** Технологии разработки программного обеспечения
****Студент:**** Черноусов Олег Андреевич
****Группа:**** еозИСИП - 3030
****Тема:**** Система учёта заявок

1. Цель работы
Научиться выделять слои, компоненты и границы ответственности программной системы.

2. Ход работы

****2.1. Выбор архитектуры.****
На основе требований из лабораторной работы №1 выбрана слоистая архитектура: presentation/controller → service → repository/data access → database. Вариант рекомендован методичкой и подходит для учебного проекта: обеспечивает разделение ответственности, тестируемость и простоту сопровождения.

****2.2. Слои системы.****

Слой	**Ответственность**	**Не должен делать**
Controller (Presentation)	Принимает запросы, валидирует входные данные, передаёт в service, возвращает результат	Не содержит SQL, не реализует бизнес-правила
Service	Реализует бизнес-правила: смена статусов, назначение исполнителя, проверка прав	Не рисует интерфейс, не работает с БД напрямую
Repository (Data Access)	Читает и изменяет данные, выполняет запросы к БД, маппит строки в объекты	Не принимает бизнес-решений
Database	Хранит данные, обеспечивает целостность	Не содержит UI-логики и бизнес-правил

****2.3. Основные модули.****
Пользователи (users), заявки (requests), статусы (statuses), категории (categories), комментарии (comments).

****2.4. Структура каталогов исходного кода.****
```
src/
├── presentation/ # контроллеры, интерфейс
│ └── controllers/
├── service/ # бизнес-логика
```

│ ├── request_service.py
│ ├── user_service.py
│ └── category_service.py
├── repository/ # доступ к данным
│ ├── request_repository.py
│ ├── user_repository.py
│ └── category_repository.py
├── models/ # сущности (User, Request, Status, Category, Comment)
├── database/ # подключение к БД, схема
└── main.py # точка входа

2.5. Ключевые классы и модули.

Класс/модуль	Назначение	Входные данные	Результат
RequestService	Бизнес-логика заявок: создание, назначение, смена статуса	Данные заявки, id исполнителя, новый статус	Обновлённая заявка, проверка прав
RequestRepository	Чтение и запись заявок в БД	id заявки, фильтры	Объект заявки или список заявок
UserService	Аутентификация и проверка ролей	Логин, пароль, роль	Пользователь, права доступа
CategoryRepository	Справочник категорий	—	Список категорий
CommentService	Добавление комментариев к заявке	id заявки, текст, автор	Комментарий

2.6. Диаграммы.
Архитектурная диаграмма (слои) — `docs/architecture/layers.png`. Диаграмма компонентов — `docs/architecture/components.png`.

2.7. Обоснование выбора архитектуры.
За: чёткое разделение ответственности; каждый слой тестируется отдельно; легко расширять (например, заменить БД или добавить новый интерфейс); соответствует учебному проекту и требованиям методички. **Против:** больше файлов и «церемоний», чем в простом монолите; для маленькой системы часть слоёв может показаться избыточной. Для проекта «Система учёта заявок» выбранная архитектура оптимальна: она даёт структуру, которую легко защитить и развивать в следующих лабораторных.

2.8. Работа с нейросетью.
Нейросеть использовалась для генерации двух вариантов архитектуры и их сравнения, а также как архитектурный ревьюер. Из ответа ИИ взята слоистая архитектура как основа; замечания ревьюера разобраны: часть принята, часть отклонена как избыточная для учебного проекта.

3. AI-журнал

Промпт 1 (варианты архитектуры):
«Ты — архитектор ПО. Вот требования к системе учёта заявок: 4 роли (администратор, оператор, исполнитель, заявитель), заявки со статусами (Новая, В работе, Выполнена, Закрыта), категории, комментарии, отчёты. Предложи минимум 2 архитектурных варианта и сравни их по критериям: простота, тестируемость, расширяемость, стоимость внедрения, соответствие учебному проекту. Формат: таблица сравнения + краткий вывод.»

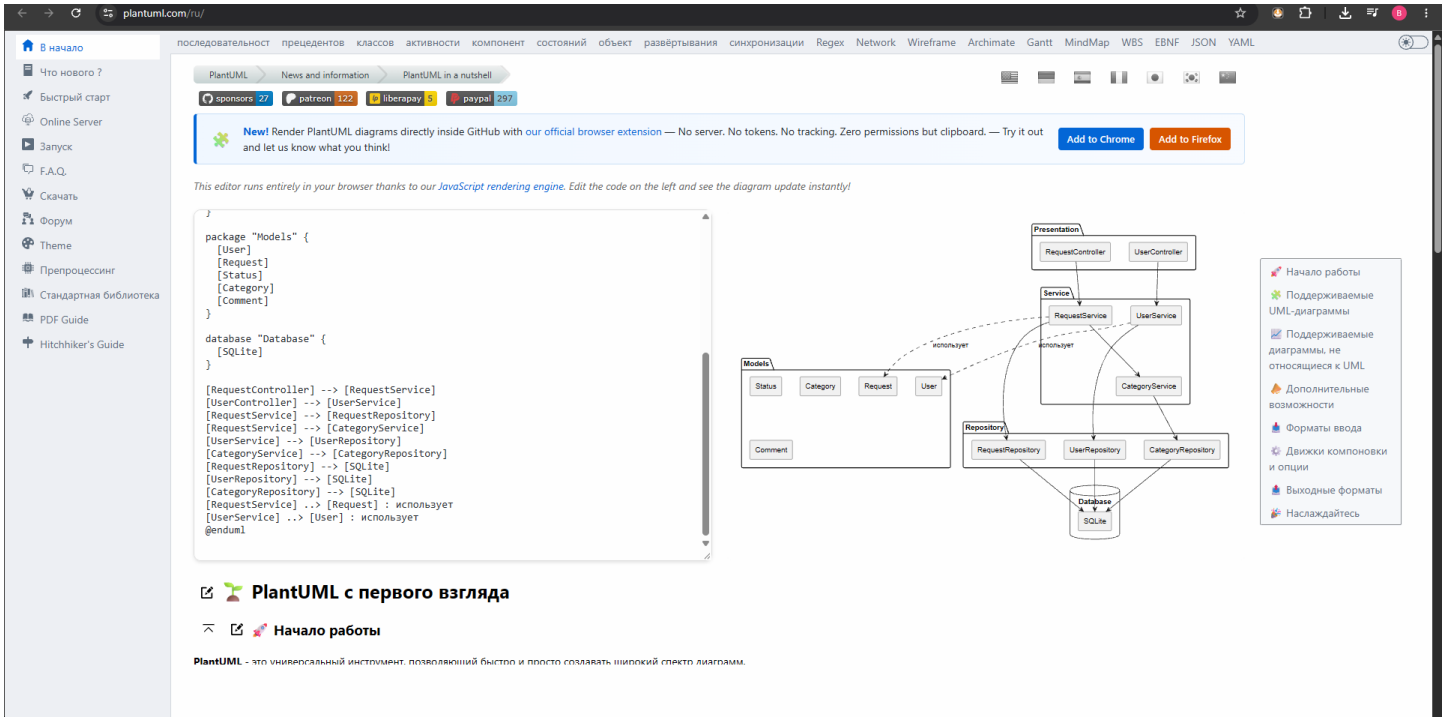
Промпт 2 (ревью):
«Ты — архитектурный ревьюер. Вот моя слоистая архитектура: Presentation (Controller) → Service → Repository → Database; модули: users, requests, statuses, categories, comments; структура каталогов: presentation/, service/, repository/, models/, database/. Найди нарушения разделения ответственности, слишком связанные компоненты и потенциальные проблемы. Формат: список замечаний.»

Запись AI-журнала:

1. **Задача:** получить и сравнить варианты архитектуры для системы учёта заявок, затем провести ревью собственной диаграммы.
2. **Модель:** DEEPSEEK-v3.
3. **Промпт:** Ты — архитектурный ревьюер. Вот моя слоистая архитектура: Presentation (Controller) → Service → Repository → Database; модули: users, requests, statuses, categories, comments; структура каталогов: presentation/, service/, repository/, models/, database/. Найди нарушения разделения ответственности, слишком связанные компоненты и потенциальные проблемы. Формат: список замечаний.».
4. **Почему такой промпт:** задал роль (архитектор, ревьюер), контекст требований, критерии сравнения и формат — чтобы получить структурированный, проверяемый ответ.
5. **Почему такая модель:** выбрана за качество структурных ответов и работу с длинным контекстом.
6. **Результат ИИ:** два варианта архитектуры (слоистая и монолитная/модульная), таблица сравнения, список замечаний ревьюера.
7. **Проверка:** сравнил варианты с требованиями методички; проверил, что в моей диаграмме каждый слой выполняет только свою работу; сверил структуру каталогов с ТЗ.
8. **Собственная доработка:** выбрал слоистую архитектуру самостоятельно; из замечаний ревьюера принял «не допускать SQL в контроллере» и «не смешивать бизнес-логику и доступ к данным»; отклонил предложение добавить кэширование как избыточное для учебного проекта; исправил структуру каталогов, добавив `models/` и `database/`.

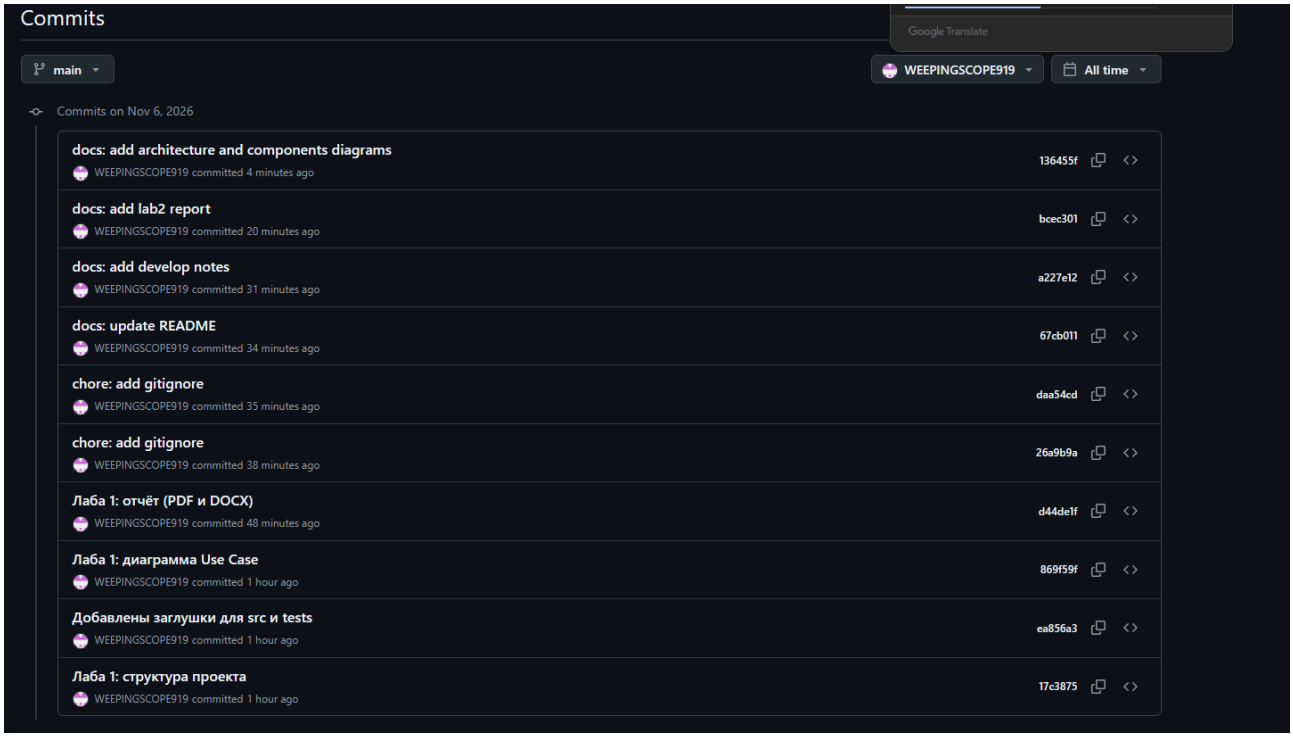
9. **Вывод:** ИИ полезен для генерации вариантов и критики, но выбор архитектуры и её обоснование остаются за разработчиком; замечания ревьюера нужно фильтровать по контексту проекта.

4. Вывод
В ходе работы освоено проектирование слоистой архитектуры: выделены 4 слоя, определены их ответственность и границы, описаны модули и структура каталогов, построены архитектурная диаграмма и диаграмма компонентов. Выбранная архитектура обоснована и соответствует требованиям проекта. Диаграммы размещены в `docs/architecture/`, проект обновлён в Git-репозитории.

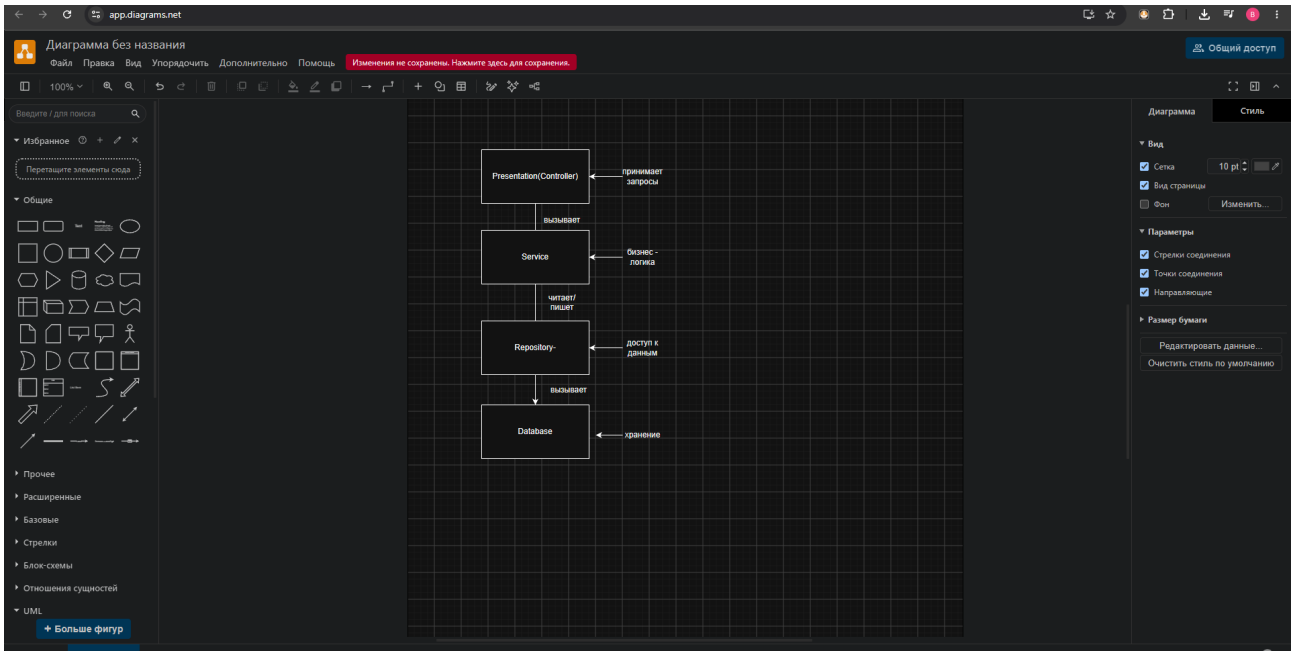


Скриншоты к работе:

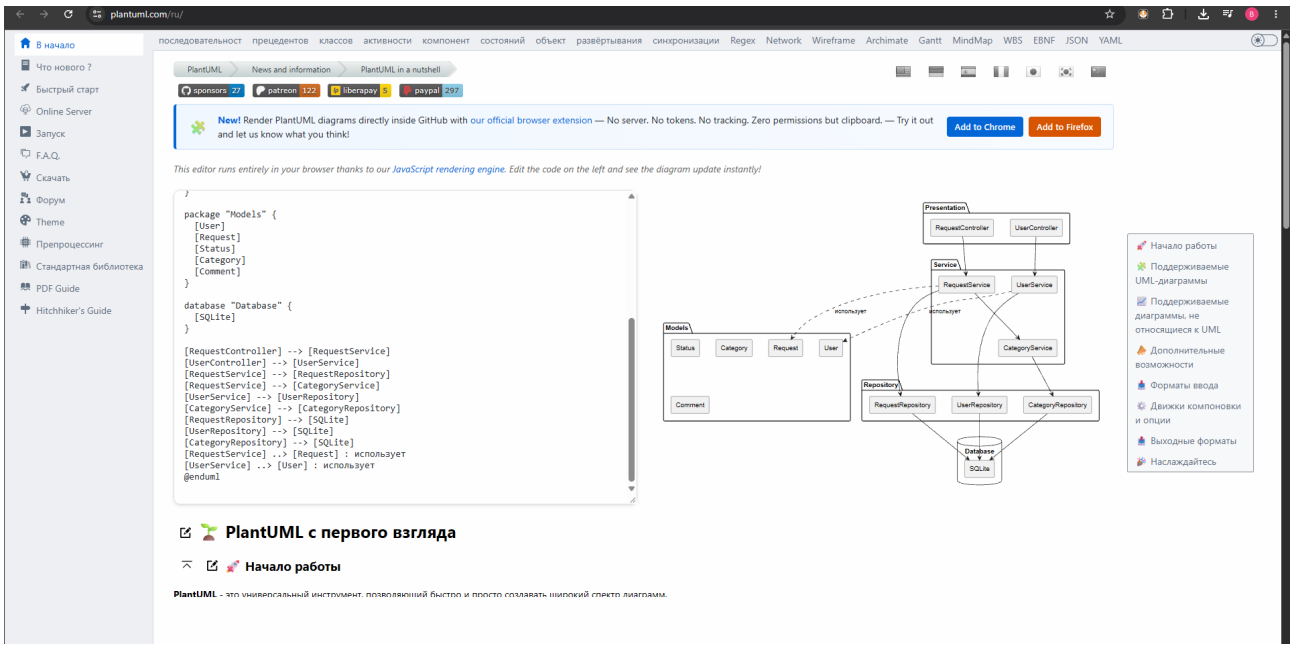
1.Коммиты



2.Работа с диаграммой в https://app.diagrams.net/



3.Работа с диаграммой https://plantuml.com (тест создания диаграммы через код)



4.выгрузка с баша на гитхаб

